

Understanding Cross Entropy Loss

1 1. What is a Loss Function?

A loss function measures how wrong the model prediction is.

Main idea:

Prediction $\rightarrow$ Compare with Truth $\rightarrow$ Calculate Error

Purpose:

Minimize Loss

Interpretation:

- Low loss = better prediction
- High loss = worse prediction

Training goal: Make the loss as small as possible.

2 2. Loss Function Used in the Code

Code:

```
loss_fn = nn.CrossEntropyLoss(weight=weights)
```

This means:

The model is solving a classification problem.

Cross Entropy Loss is used to compare:

- predicted class scores
- actual class label

3 3. What Input Does It Expect?

Cross Entropy Loss needs two inputs:

Input 1: Model Prediction

Expected shape:

(batch_size, num_classes)

Example:

```
[  
  [2.4, 0.8, 1.1],  
  [0.3, 2.7, 0.5]  
]
```

Important:

These are logits.

Not probabilities.

Do NOT apply softmax.

PyTorch handles it internally.

Input 2: True Labels

Expected shape:

(batch_size)

Example:

```
[0,1]
```

Meaning:

- First sample belongs to class 0
- Second sample belongs to class 1

Required type:

LongTensor

4. How It Works Internally

Step 1:

Convert logits into probabilities using Softmax

$$P(y_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

Step 2:

Pick the probability of the correct class

Step 3:

Take negative log

$$Loss = -\log(P(correct))$$

Flow:

$$Logits \rightarrow Softmax \rightarrow CorrectProbability \rightarrow Log \rightarrow Loss$$

5 5. Mathematical Formula

Cross Entropy formula:

$$L = -\log(p)$$

Where:

- L = loss
- p = probability of correct class

Core formula visualization:

$$L = -\log(p)$$

6 6. Practical Example

Suppose:

Three classes:

- Cat
- Dog
- Bird

Model prediction:

$$[0.1, 0.8, 0.1]$$

True label:

Dog

Correct probability:

$$p = 0.8$$

Loss:

$$L = -\log(0.8) = 0.223$$

Small loss.

Good.

Now:

Prediction:

[0.7, 0.2, 0.1]

True label:

Dog

Correct probability:

$$p = 0.2$$

Loss:

$$L = -\log(0.2) = 1.609$$

Large loss.

Bad.

7 7. What Does It Return?

Output:

Single scalar value.

Example:

0.42

Type:

Float tensor.

Example:

`tensor(0.42)`

To get normal number:

`loss.item()`

8 8. Range of Output Values

Range:

$$0 \leq Loss < \infty$$

Minimum:

0

Maximum:

Unlimited.

Important:

- Lower is better
- Higher is worse

9 9. Which Values are Good?

Loss Value	Interpretation
> 2	Very poor prediction
$1 - 2$	Weak prediction
$0.5 - 1$	Average prediction
$0.1 - 0.5$	Good prediction
< 0.1	Excellent prediction

Note:

Always check accuracy with loss.

10 10. Why weight=weights?

Used for class imbalance.

Example:

- Class A = 1000 samples
- Class B = 100 samples

Without weights:

Model focuses more on Class A.

With weights:

Rare class gets more importance.

Example:

`weights=[1,4]`

Meaning:

Mistakes in Class B are penalized 4 times more.

11 11. Training Flow in Your Code

Code flow:

```
outputs = model(images)
loss = loss_fn(outputs, labels)
loss.backward()
optimizer.step()
```

Meaning:

- Model predicts
- Loss measures error
- Backpropagation computes gradients
- Optimizer updates weights

12 12. What Good Training Looks Like

Good training:

$$2.1 \rightarrow 1.4 \rightarrow 0.9 \rightarrow 0.4$$

Loss decreasing.

Good.

Bad training:

$$0.9 \rightarrow 1.5 \rightarrow 2.0$$

Loss increasing.

Problem.

13 13. Common Mistakes

- Applying softmax before CrossEntropyLoss
- Wrong target shape
- Wrong target type
- Wrong class indexing

14 14. Final Summary

Cross Entropy Loss:

- Used for classification
- Input = logits + true labels
- Output = single loss value
- Range = 0 to infinity
- Lower value = better
- Class weights help imbalance

Final goal:

Minimize Loss